

```
#importing the librararies
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib as mpl
import matplotlib.pyplot as plt
import pmdarima as pm
```

```
#Loading the dataset
df= pd.read_csv("C:/Users/saiha/Desktop/Analytics_T5/Project/TM_H1.csv")
```

```
df = df.dropna(axis=1)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 40 entries, 0 to 39
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Quarter    40 non-null      object
1   Net Sales   40 non-null      int64
dtypes: int64(1), object(1)
memory usage: 772.0+ bytes
```

```
df.head(5)
```

	Quarter	Net Sales
0	30-09-2025	12751
1	30-06-2025	15546
2	31-03-2025	19869
3	31-12-2024	17040
4	30-09-2024	15518

```
df.rename(columns={'Quarter': 'Date'}, inplace=True)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 40 entries, 0 to 39
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Date        40 non-null      object
1   Net Sales   40 non-null      int64
dtypes: int64(1), object(1)
memory usage: 772.0+ bytes
```

```
df['Date'] = pd.to_datetime(df['Date'], format = 'mixed')
```

```
# Setting the date column as the index, check whether the date column is datetime64 datatype otherwise change it
df = df.set_index(df['Date']).drop('Date', axis=1)
```

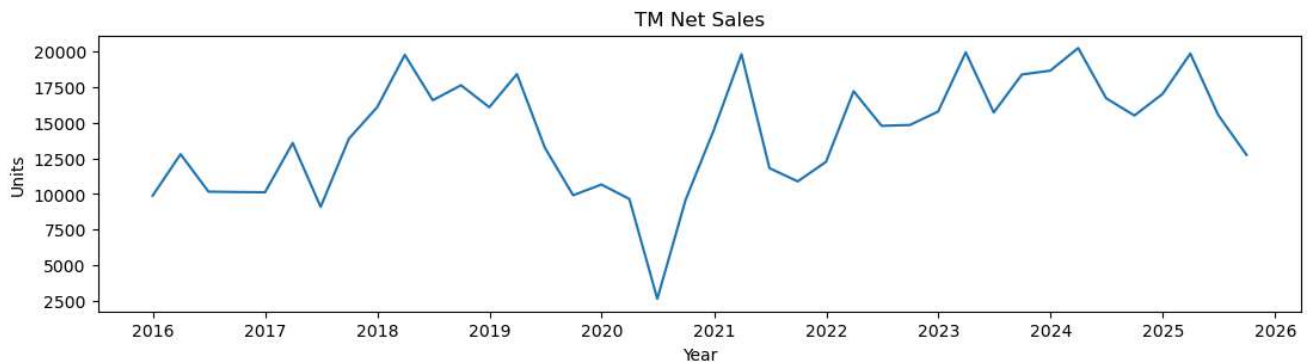
```
df.head()
```

	Net Sales
Date	
2025-09-30	12751
2025-06-30	15546
2025-03-31	19869
2024-12-31	17040
2024-09-30	15518

```
# Descriptive statistics
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
Net Sales	40.0	14311.65	3935.61249	2634.0	10834.5	14821.5	17086.0	20260.0

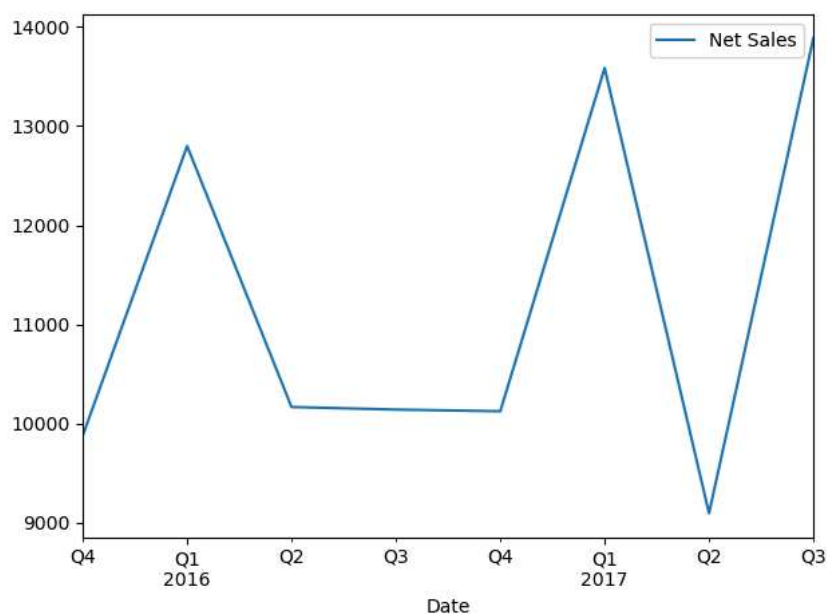
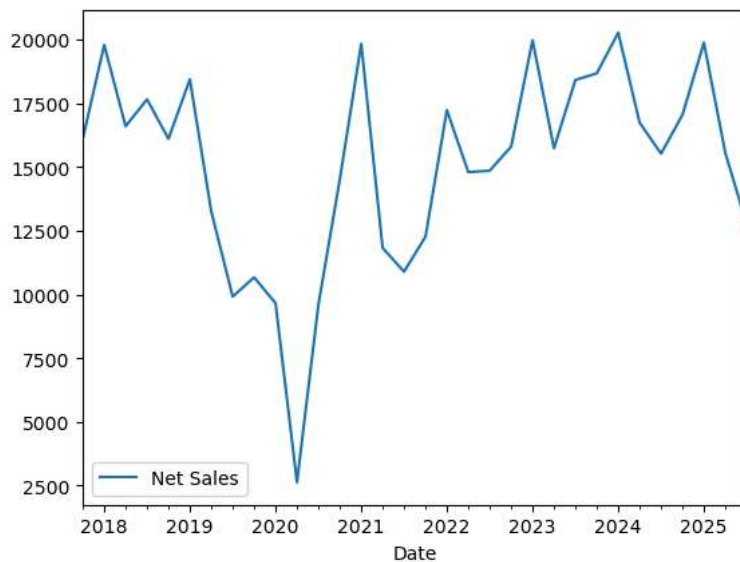
```
# Plotting the TS
plt.figure(figsize =(13,3))
plt.title("TM Net Sales")
plt.xlabel('Year')
plt.ylabel('Units')
plt.plot(df['Net Sales'])
plt.show()
```



```
from statsmodels.tsa.stattools import adfuller
adf_result = adfuller(df['Net Sales'])
print(f"ADF Statistic: {adf_result[0]:.2f}")
print(f"p-value: {adf_result[1]:.2f}")
```

```
ADF Statistic: -2.52
p-value: 0.11
```

```
# Splitting the dataset into training and testing datasets 90% training and 10% testing data
import numpy as np
from sklearn.model_selection import train_test_split
df_train, df_test = train_test_split(df, train_size=0.80, shuffle = False)
df_train.plot()
df_test.plot()
plt.tight_layout()
plt.show()
```



✓ !pip install pmdarima

```
# Model development using df_train dataset
from pmdarima.arima import auto_arima
auto_arima_model = auto_arima(df_train, start_p=0, d=1, start_q=0, max_p=5, max_d=5, max_q=5, start_P=0, D=1, start_Q=0, test='adf',
max_P=5, max_D=5, Max_Q=5, m=4, seasonal = True, trace=True, error_action='warn', suppress_warnings=True, stepwise=True,)
auto_arima_model_fit = auto_arima_model.fit(df_train)
```

```
Performing stepwise search to minimize aic
ARIMA(0,1,0)(0,1,0)[4] : AIC=521.285, Time=0.01 sec
ARIMA(1,1,0)(1,1,0)[4] : AIC=518.157, Time=0.04 sec
ARIMA(0,1,1)(0,1,1)[4] : AIC=512.084, Time=0.08 sec
ARIMA(0,1,1)(0,1,0)[4] : AIC=522.133, Time=0.02 sec
ARIMA(0,1,1)(1,1,1)[4] : AIC=514.082, Time=0.17 sec
ARIMA(0,1,1)(0,1,2)[4] : AIC=514.082, Time=0.07 sec
ARIMA(0,1,1)(1,1,0)[4] : AIC=515.885, Time=0.07 sec
ARIMA(0,1,1)(1,1,2)[4] : AIC=inf, Time=0.18 sec
ARIMA(0,1,0)(0,1,1)[4] : AIC=513.191, Time=0.03 sec
ARIMA(1,1,1)(0,1,1)[4] : AIC=inf, Time=0.13 sec
ARIMA(0,1,2)(0,1,1)[4] : AIC=inf, Time=0.08 sec
ARIMA(1,1,0)(0,1,1)[4] : AIC=513.457, Time=0.06 sec
ARIMA(1,1,2)(0,1,1)[4] : AIC=inf, Time=0.16 sec
ARIMA(0,1,1)(0,1,1)[4] intercept : AIC=513.820, Time=0.11 sec
```

```
Best model: ARIMA(0,1,1)(0,1,1)[4]
Total fit time: 1.216 seconds
```

```
# predicting test data set values
n_periods = len(df_test)
test_predictions = pd.DataFrame(auto_arima_model_fit.predict(n_periods=n_periods))
test_predictions.columns=['Test Pred']
```

```
C:\Users\saiha\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:837: ValueWarning: No supported index is available.
    return get_prediction_index(
C:\Users\saiha\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:837: FutureWarning: No supported index is available
    return get_prediction_index(
```

```
test_predictions
```

	Test Pred
32	15564.345564
33	15030.734508
34	19655.267472
35	16877.956178
36	16542.894220
37	16009.283163
38	20633.816127
39	17856.504833

```
# setting the index for the test predictions
test_predictions.index = df_test.index
test_predictions.index
```

```
DatetimeIndex(['2017-09-30', '2017-06-30', '2017-03-31', '2016-12-31',
               '2016-09-30', '2016-06-30', '2016-03-31', '2015-12-31'],
              dtype='datetime64[ns]', name='Date', freq=None)
```

```
test_predictions['Test Pred'] = pd.DataFrame(test_predictions['Test Pred'], index=test_predictions.index)
test_predictions['Test Pred']
```

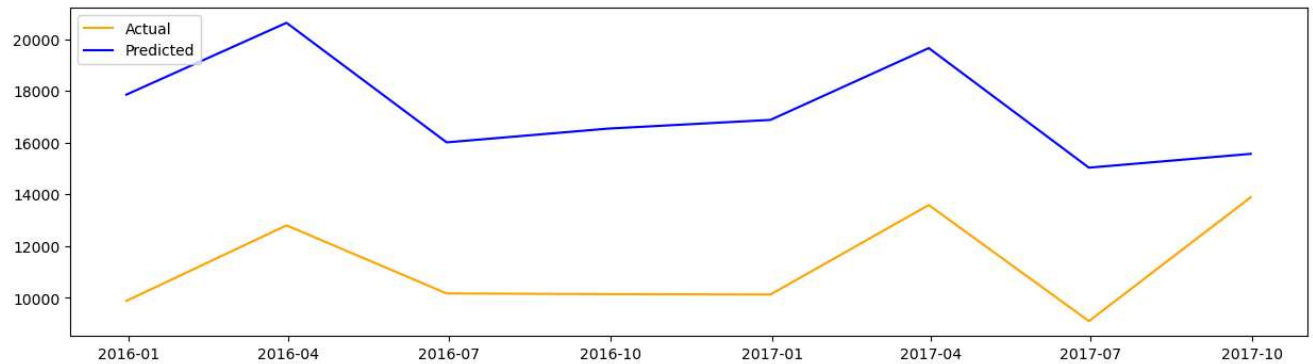
Date	
2017-09-30	15564.345564
2017-06-30	15030.734508
2017-03-31	19655.267472
2016-12-31	16877.956178
2016-09-30	16542.894220
2016-06-30	16009.283163
2016-03-31	20633.816127
2015-12-31	17856.504833

Name: Test Pred, dtype: float64

```
df_test['Test Pred'] = test_predictions.round(2)
df_test.head()
```

	Net Sales	Test Pred
2017-09-30	13887	15564.35
2017-06-30	9094	15030.73
2017-03-31	13586	19655.27
2016-12-31	10123	16877.96
2016-09-30	10140	16542.89

```
# Plotting the test data actuals and predictions
plt.figure(figsize=(15,4))
plt.plot(df_test['Net Sales'], color='Orange', label='Actual')
plt.plot(df_test['Test Pred'], color='blue', label='Predicted')
plt.legend(loc="upper left")
plt.show()
```



```
future_forecast = auto_arima_model.predict(n_periods=12)
future_forecast = pd.DataFrame(future_forecast, columns=['Future Forecast'])
```

```
C:\Users\saiha\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:837: ValueWarning: No supported index is available.
    return get_prediction_index(
C:\Users\saiha\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:837: FutureWarning: No supported index is available
    return get_prediction_index(
```

```
# Predicting 12 months future values, creating the index
future_forecast.index = pd.date_range(start='2025-09-01', periods= 12, freq='MS')
```

```
future_forecast.index
```

```
DatetimeIndex(['2025-09-01', '2025-10-01', '2025-11-01', '2025-12-01',
                '2026-01-01', '2026-02-01', '2026-03-01', '2026-04-01',
                '2026-05-01', '2026-06-01', '2026-07-01', '2026-08-01'],
              dtype='datetime64[ns]', freq='MS')
```

```
# Predicting next 12 months and storing them in a dataframe, naming the column of the dataframe
Futurepredicted_values = pd.DataFrame(auto_arima_model_fit.predict(12))
Futurepredicted_values.columns = ['Future Forecast']
Futurepredicted_values
```

```
C:\Users\saiha\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:837: ValueWarning: No supported index is available.
    return get_prediction_index(
C:\Users\saiha\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:837: FutureWarning: No supported index is available
    return get_prediction_index(
```

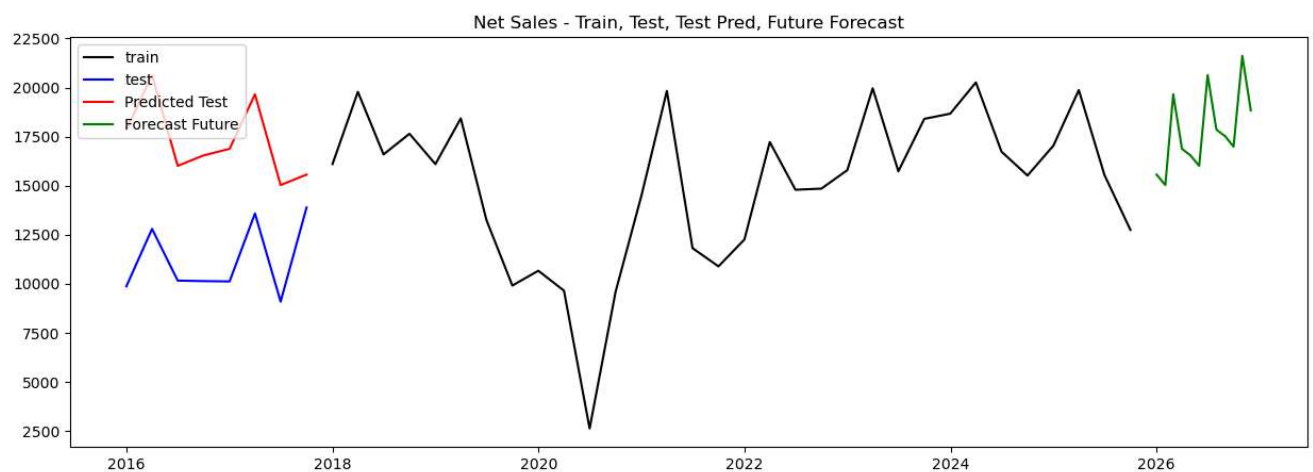
	Future Forecast
32	15564.345564
33	15030.734508
34	19655.267472
35	16877.956178
36	16542.894220
37	16009.283163
38	20633.816127
39	17856.504833
40	17521.442875
41	16987.831819
42	21612.364783
43	18835.053489

```
# Creating indices with future dates for the future predicted values
Futurepredicted_values.index = pd.date_range(start='2026-01-01', periods= 12, freq='MS')
Futurepredicted_values
```

Future Forecast

2026-01-01	15564.345564
2026-02-01	15030.734508
2026-03-01	19655.267472
2026-04-01	16877.956178
2026-05-01	16542.894220
2026-06-01	16009.283163
2026-07-01	20633.816127
2026-08-01	17856.504833
2026-09-01	17521.442875
2026-10-01	16987.831819
2026-11-01	21612.364783
2026-12-01	18835.053489

```
plt.figure(figsize =(15,5))
plt.plot(df_train['Net Sales'], color='black', label='train')
plt.plot(df_test['Net Sales'], color='blue', label='test')
plt.plot(df_test['Test Pred'], color='red', label='Predicted Test')
plt.plot(Futurepredicted_values['Future Forecast'], color='green', label='Forecast Future')
plt.title("Net Sales - Train, Test, Test Pred, Future Forecast")
plt.legend(loc="upper left")
plt.show()
```



```
# evaluate the model - df_test['production'], df_test['testpredictions']
from sklearn.metrics import mean_absolute_error, mean_squared_error, root_mean_squared_error, r2_score
mae = mean_absolute_error(df_test['Net Sales'], df_test['Test Pred'])
mse = mean_squared_error(df_test['Net Sales'], df_test['Test Pred'])
rmse = root_mean_squared_error(df_test['Net Sales'], df_test['Test Pred'])
r_squared = r2_score(df_test['Net Sales'], df_test['Test Pred'])

print('R-squared value: %.3f' % r_squared) # r-squared will be negative when the model is good and need to recheck the parameters
```